

1 Группировка сложных селекторов

Пусть у нас есть два следующих селектора:

```
.block h2 {  
    color: red;  
}  
.block h3 {  
    color: red;  
}
```

Так как эти селекторы делают одно и тоже их можно сгруппировать через запятую:

```
.block h2, .block h3 {  
    color: red;  
}
```

Ошибка новичков

Все, что разделено запятой при группировке, должно представлять собой полноценный селектор. Давайте посмотрим, где здесь можно ошибиться.

Пусть у нас есть вот такой код:

```
#block h2 {  
    color: red;  
}  
#block h3 {  
    color: red;  
}
```

Давайте сгруппируем два селектора в один. Вот правильный вариант группировки:

```
#block h2, #block h3 {  
    color: red;  
}
```

А вот неправильный вариант группировки:

```
#block h2, h3 {  
    color: red;  
}
```

В этом неправильном варианте группировки новичкам почему-то кажется, что `#block` относится и к `h2`, и к `h3`, стоящему после запятой. Но селекторы не

действуют через запятую и фактически этот неправильный вариант группировки можно переписать вот так:

```
#block h2 {  
    color: red;  
}  
h3 {  
    color: red;  
}
```

Сравните с изначальным вариантом, который мы начали сокращать:

```
#block h2 {  
    color: red;  
}  
#block h3 {  
    color: red;  
}
```

Практические задачи

Задача 1

Упростите код, используя группировку селекторов:

```
#block h1 {  
    text-align: center;  
}  
#block h2 {  
    text-align: center;  
}
```

Задача 2

Упростите код, используя группировку селекторов:

```
#block h1 {  
    text-align: center;  
    font-size: 30px;  
}  
#block h2 {  
    text-align: center;  
    font-size: 20px;  
}
```

Задача 3

Упростите код, используя группировку селекторов:

```
#block h1 {  
    text-align: center;  
}  
#block h2 {  
    text-align: center;  
    color: blue;  
}  
#block h3 {  
    text-align: center;  
    font-size: 16px;  
    color: red;  
}
```

Задача 4

Упростите код, используя группировку селекторов:

```
#block h1.eee {  
    text-align: center;  
    font-size: 30px;  
}  
#block h2.zzz {  
    text-align: center;  
    font-size: 20px;  
}
```

Задача 5

Упростите код, используя группировку селекторов:

```
.xxx .kkk {  
    text-align: center;  
}  
.eee h2.zzz {  
    text-align: center;  
}  
#eee h2 {  
    text-align: center;  
}
```

Задача 6

Упростите код, используя группировку селекторов:

```
.eee h2.zzz {  
    text-align: center;  
}  
.xxx {  
    text-align: center;  
}
```

2 Практика на комбинации селекторов

Задача 1

Напишите селектор, который выберет абзацы, расположенные внутри дивов `div`. Проверьте ваш селектор на следующем коде:

```
<div>  
    <p>  
        +++  
    </p>  
    <p>  
        +++  
    </p>  
</div>  
<div>  
    <p>  
        +++  
    </p>  
</div>  
  
<p>  
    ---  
</p>  
<p>  
    ---  
</p>
```

Задача 2

Напишите селектор, который выберет все `h2`, расположенные внутри дивов `div`. Проверьте ваш селектор на следующем коде:

```
<div>
  <h2>+++</h2>
  <p>
    ---
  </p>
  <p>
    ---
  </p>
</div>
<div>
  <h2>+++</h2>
  <p>
    ---
  </p>
</div>

<h2>---</h2>
<p>
  ---
</p>
```

Задача 3

Напишите селектор, который выберет все абзацы `p` из элемента с `id`, равным `block`. Проверьте ваш селектор на следующем коде:

```
<div id="block">
  <h2>---</h2>
  <p>
    +++
  </p>
  <p>
    +++
  </p>
</div>
<div>
  <h2>---</h2>
  <p>
    ---
  </p>
```

```
</div>
```

```
<h2>---</h2>
```

```
<p>
```

```
---
```

```
</p>
```

Задача 4

Напишите селектор, который выберет все `h2` из элемента с `id`, равным `block`. Проверьте ваш селектор на следующем коде:

```
<div id="block">
```

```
  <h2>+++</h2>
```

```
  <p>
```

```
    ---
```

```
  </p>
```

```
  <p>
```

```
    ---
```

```
</p>
```

```
  <h2>+++</h2>
```

```
  <p>
```

```
    ---
```

```
</p>
```

```
</div>
```

```
<div>
```

```
  <h2>---</h2>
```

```
  <p>
```

```
    ---
```

```
</p>
```

```
</div>
```

```
<h2>---</h2>
```

```
<p>
```

```
---
```

```
</p>
```

Задача 5

Напишите селектор, который выберет все элементы с классом `bbb`. Проверьте ваш селектор на следующем коде:

```
<div id="block">
```

```

    <h2 class="bbb">+++</h2>
    <p class="bbb">
        +++
    </p>
    <p>
        ---
    </p>

    <h2 class="bbb">+++</h2>
    <p>
        ---
    </p>
</div>
<div class="bbb">
    +++
</div>

<h2>---</h2>
<p class="bbb">
    +++
</p>

```

3 Приоритет селекторов CSS

Пусть у нас есть следующий код:

```

p {
    color: red;
}
p {
    color: green;
}

```

Как вы видите, сначала абзацам задается красный цвет, а потом ниже - зеленый. В этом случае вторая запись переопределит первую и абзацы станут зеленого цвета.

Задача 1

Расскажите, какого размера будут заголовки после выполнения следующего кода:

```

<h2>text</h2>h2 {
    font-size: 20px;
}

```

```
}  
h2 {  
    font-size: 30px;  
}
```

Правила специфичности

В предыдущем примере оба селектора были одинаковыми и имели одинаковый приоритет. Это значит, что побеждало то свойство, которое было написано ниже.

Бывают также ситуации, когда одному элементу страницы соответствуют несколько селекторов. В этом случае при конфликте свойств победит более *специфичный*, то есть более точный селектор. Давайте рассмотрим правила специфичности.

Правило первое

Класс всегда побеждает селектор тега:

```
<p class="text">text</p>p {  
    color: red;  
}  
.text {  
    color: green; /* применится этот цвет */  
}
```

Правило второе

Идентификатор всегда побеждает класс:

```
<p id="elem" class="text">text</p>.text {  
    color: red;  
}  
#elem {  
    color: green; /* применится этот цвет */  
}
```

Правило третье

При равных условиях побеждает тот селектор, у которого больше частей. Например, если у нас два селектора тегов, то победит тот, у которого тегов больше:

```
<div>
  <p>text</p>
</div>p {
  color: red;
}
div p {
  color: green; /* применится этот цвет */
}
```

4 Универсальный селектор в CSS

Универсальный селектор * позволяет выбирать все элементы.

Пример 1

Давайте выберем все элементы и покрасим их в красный цвет:

```
* {
  color: red;
}
```

Пример 2

Давайте выберем все элементы, находящиеся внутри тега div, и покрасим их в красный цвет:

```
div * {
  color: red;
}
```

Пример 3

Давайте выберем все элементы, непосредственно находящиеся внутри тега div, и покрасим их в красный цвет:

```
div > * {
  color: red;
}
```

Пример 4

Давайте выберем все элементы, находящиеся внутри тега `span`, который в свою очередь находится внутри любого тега, который в свою очередь находится внутри тега `div`:

```
div * span {  
  color: red;  
}
```

Практические задачи

Задача 1

Дан код:

```
<main>  
  <p>-</p>  
  <p>-</p>  
  
  <div>  
    <p>+</p>  
    <p>+</p>  
  </div>  
  <section>  
    <p>+</p>  
    <p>+</p>  
  </section>  
</main>
```

Напишите селектор, который выберет все абзацы, находящийся внутри любого родителя внутри тега `main`.

5 Состояния ссылок на CSS

Я думаю, что вы, посещая различные сайты в интернете, обращали внимание на то, что ссылки обычно реагируют на наведение мышкой на них. Такого эффекта можно добиться, задавая поведение ссылок в различных состояниях.

К примеру, вот так - `a:hover` - мы поймаем состояние, когда на ссылку навели курсор мышки. В этот момент мы можем, к примеру, поменять цвет ссылки или убрать/добавить ей подчеркивание.

Конструкция `:hover` называется *псевдоклассом*. Псевдоклассы позволяют отлавливать разные состояния элементов.

Кроме `:hover` есть еще псевдоклассы `:link`, которые отлавливают не посещенную ссылку, и `:visited`, которые отлавливают посещенную ссылку. На некоторых сайтах с их помощью показывают пользователям, где они были, а где - нет. Есть еще и псевдокласс `:active`, который отлавливает следующее состояние: *на элемент нажали мышкой, но еще не отпустили*.

В следующем примере для ссылки в состоянии `:hover` убирается подчеркивание, в состоянии `:link` задается красный цвет, в состоянии `:visited` - зеленый, в `:active` - голубой. В результате получится, что в начале ссылка будет красного цвета, после нажатия на нее - зеленого, если нажать на нее мышкой и не отпускать - голубого, а если навести мышкой - станет неподчеркнутой.

```
a:link {
    color: red;
}
a:visited {
    color: green;
}
a:hover {
    text-decoration: none;
}
a:active {
    color: blue;
} <a href="#">ссылка</a>
```

Результат выполнения кода:

ССЫЛКА

Задача 1

Сделайте все ссылки в состоянии `:hover` красными и неподчеркнутыми, в состоянии `:link` - голубыми, в состоянии `:visited` - зелеными, в состоянии `:active` - черными.

Обычное использование

Как правило, указываются состояния для всех типов ссылок одновременно, а потом ниже добавляются особенности поведения ссылки при наведении мышкой, вот так:

```
a{
    color: red;
}
a:hover {
    text-decoration: none;
} <a href="#">ссылка</a>
```

Задача 2

Сделайте все ссылки во всех состояниях голубыми и неподчеркнутыми, а в состоянии `:hover` - красными и подчеркнутыми.

р.с более глубоко не будет углубляться, думаю основы вы поняли и вы всегда можете посмотреть на любом доступном ресурсе значения всех свойств.если все показывать то это уже не методичка, а целый учебник. Можно например тут почитать про теги и свойства <http://htmlbook.ru/>

Так же нужно выполнять все задания по уроку где написано «задача». А в следующем уроке будет рассматривать блочную модель ..а затем и делать более продвинутые вещи